This is CS50

Week 5

Yuliia Zhukovets

Preceptor

yuliia@cs50.harvard.edu

Agenda

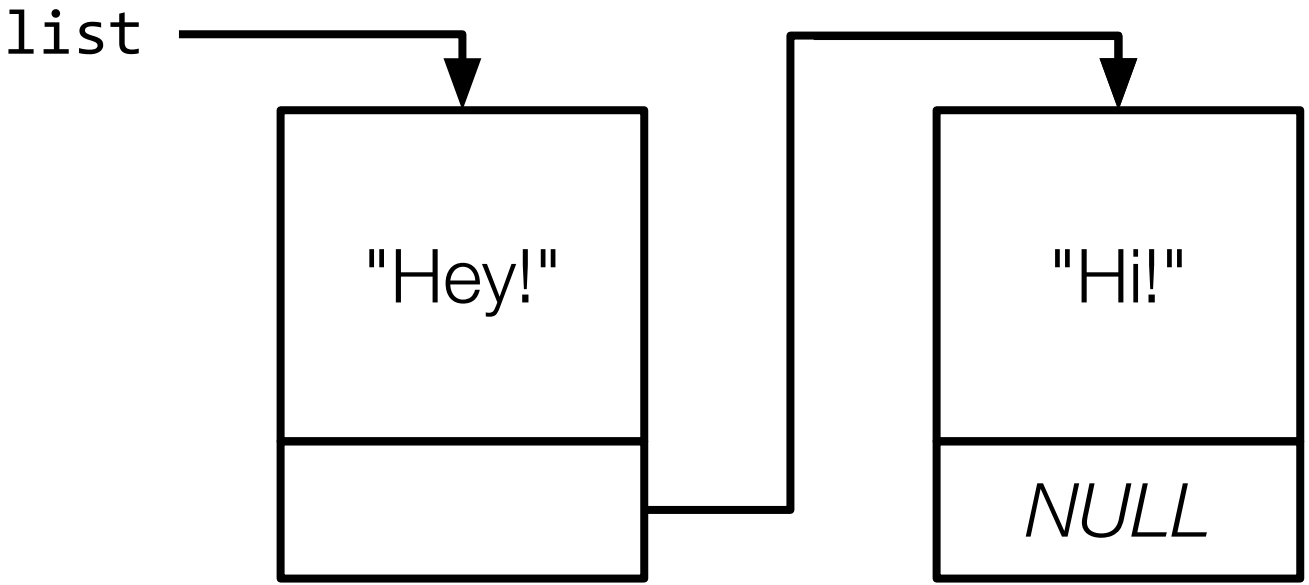
- Data Structures
- Linked Lists
- Hash Tables
- Inheritance

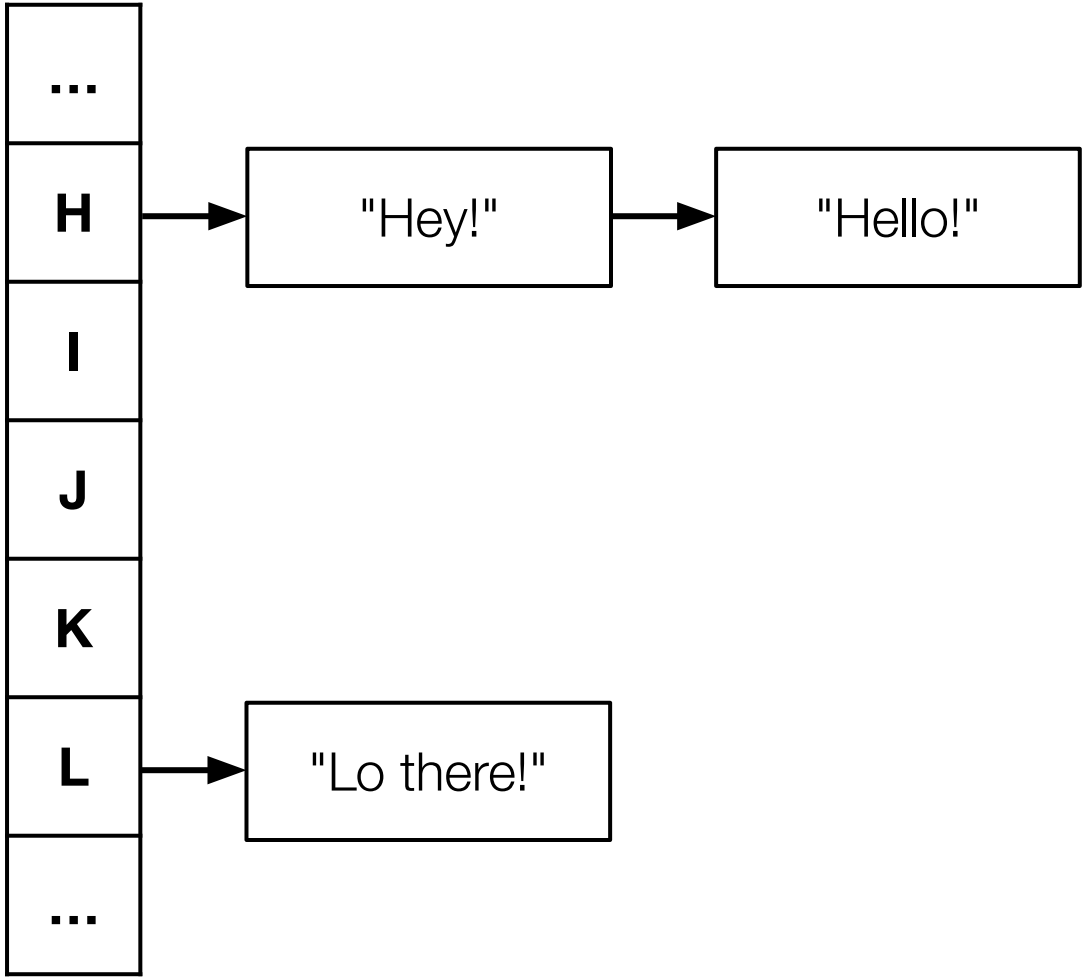
Deletion
Insertion
Search

1. Search
2. Insertion
3. Deletion

1. Insertion
2. Search
3. Deletion

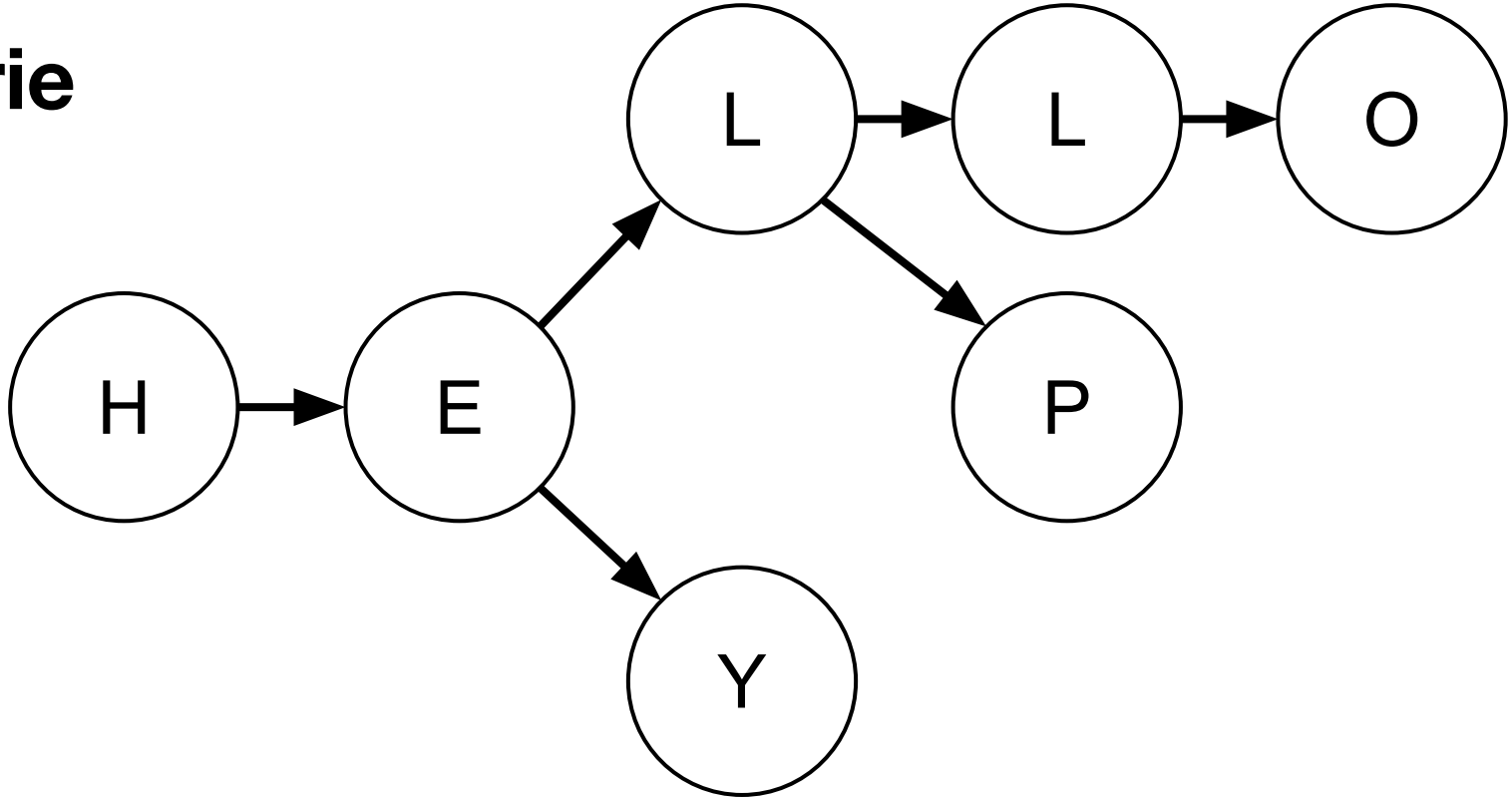
Linked List





Hash Table

Trie



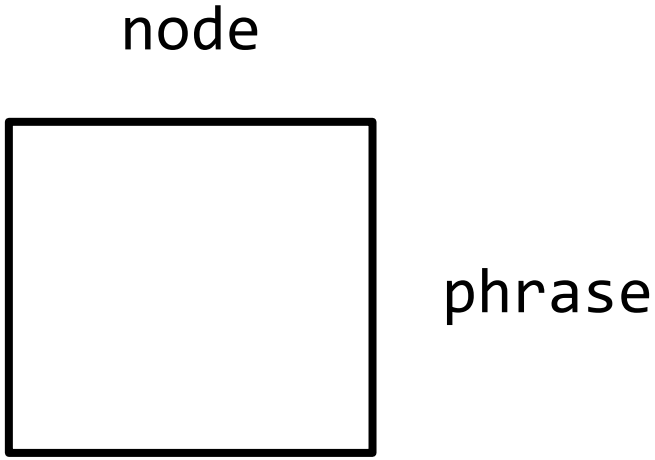
Nodes

```
typedef struct node
{
    string phrase;
    struct node *next;
}
node;
```

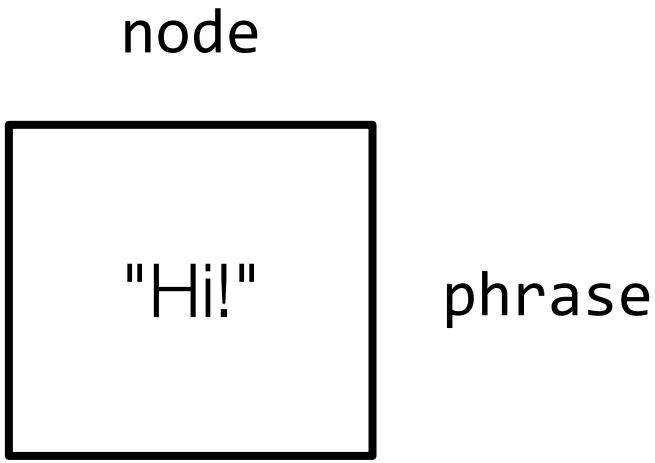
```
typedef struct node
{
    string phrase;
    struct node *next;
}
node;
```

node

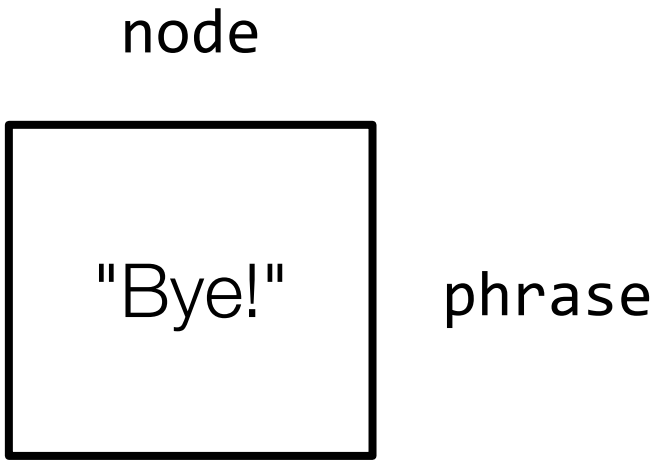
```
typedef struct node
{
    string phrase;
    struct node *next;
}
node;
```



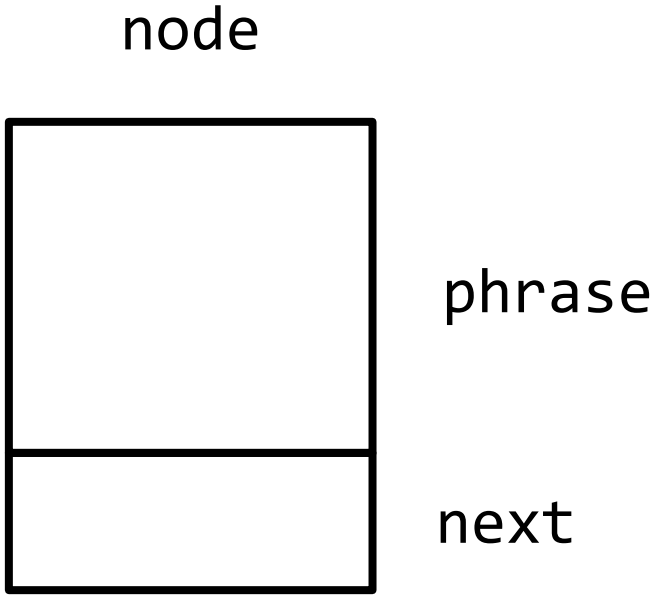
```
typedef struct node
{
    string phrase;
    struct node *next;
}
node;
```



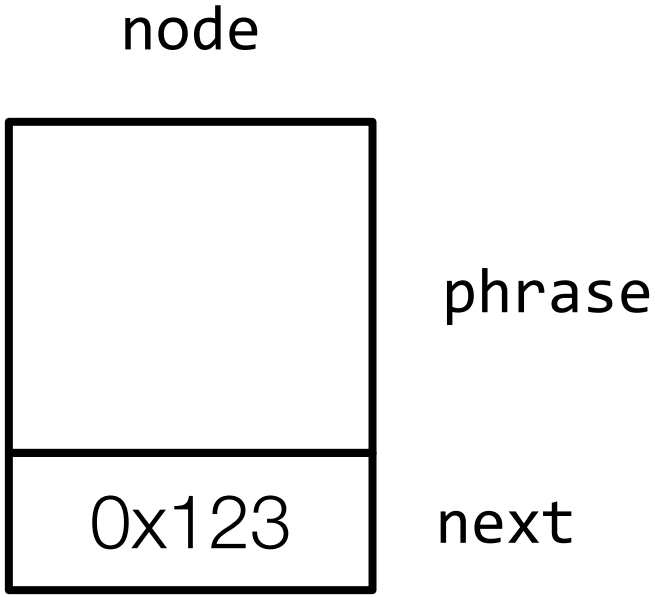
```
typedef struct node
{
    string phrase;
    struct node *next;
}
node;
```



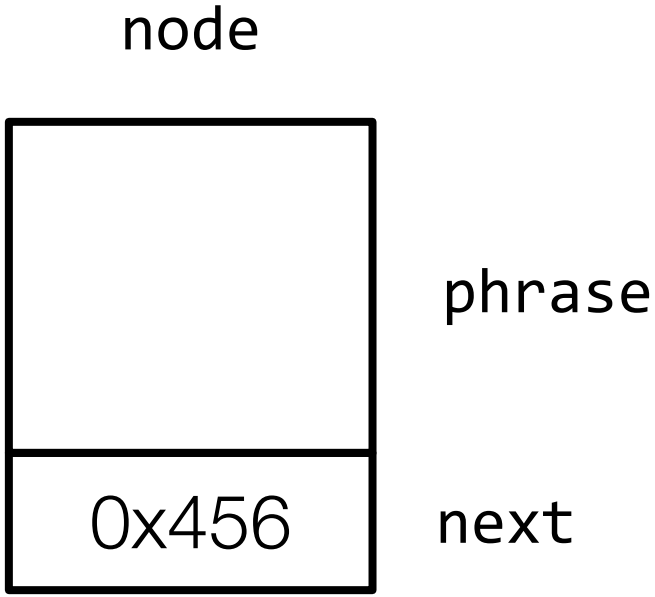
```
typedef struct node
{
    string phrase;
    struct node *next;
}
node;
```



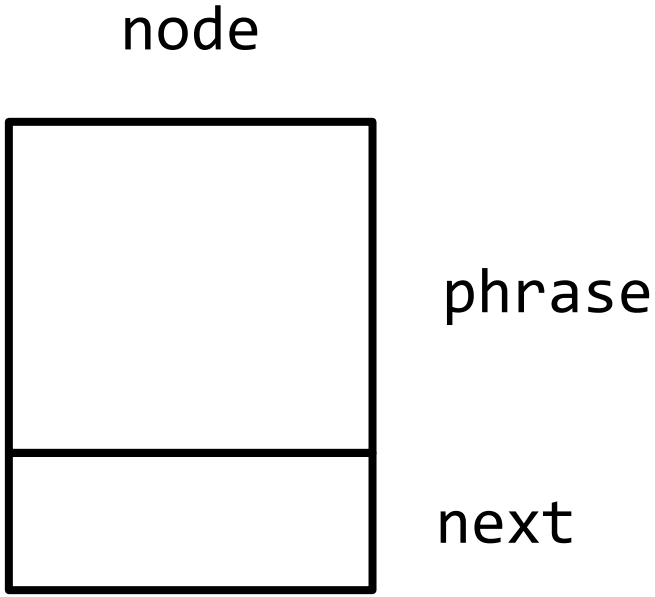
```
typedef struct node
{
    string phrase;
    struct node *next;
}
node;
```



```
typedef struct node
{
    string phrase;
    struct node *next;
}
node;
```

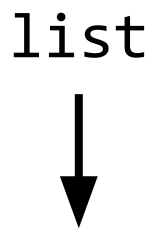


```
typedef struct node
{
    string phrase;
    struct node *next;
}
node;
```



Creating a Linked List

```
node *list = NULL;
```

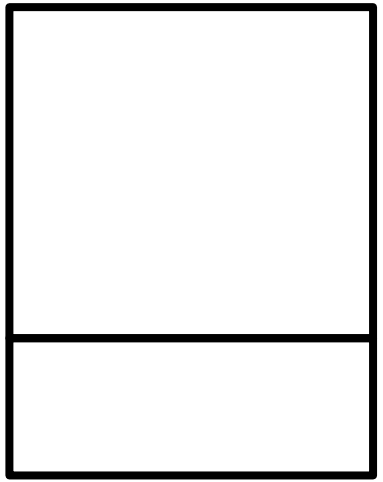


```
node *n = malloc(sizeof(node));
```

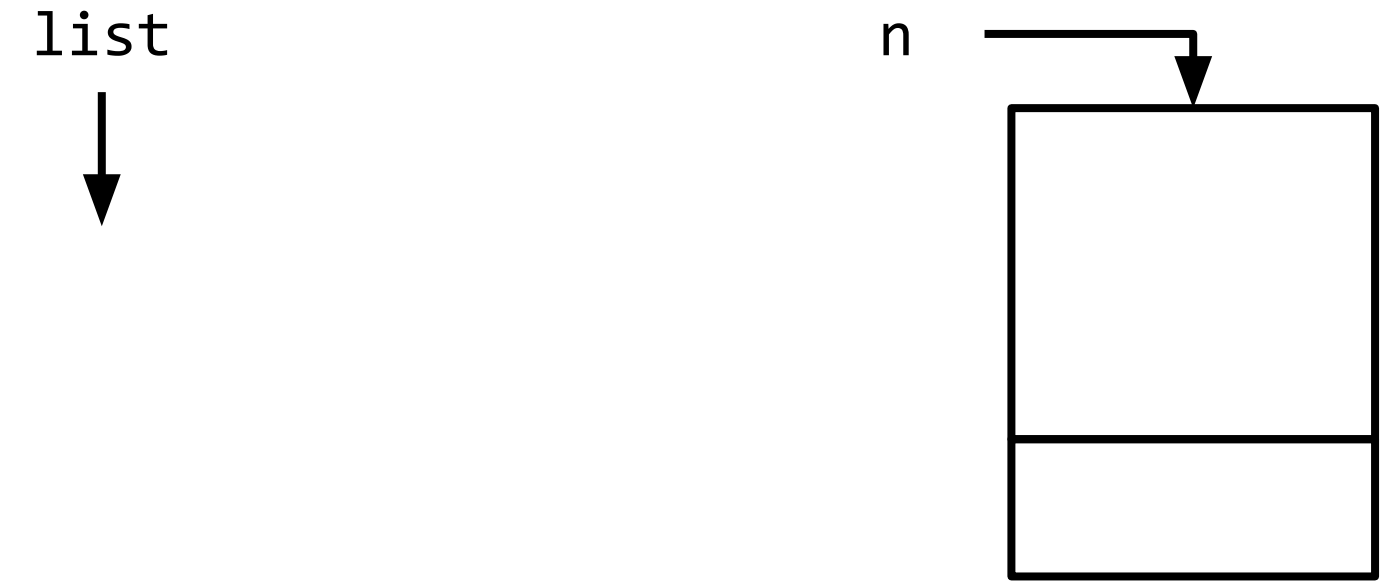
list

```
node *n = malloc(sizeof(node));
```

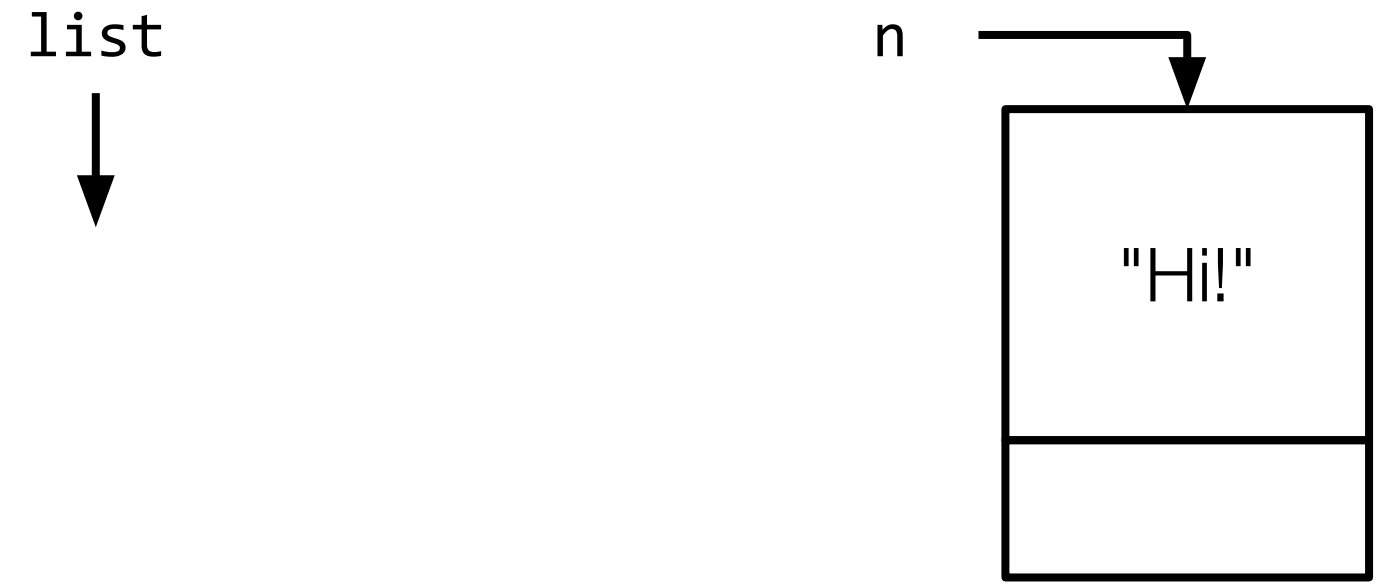
list



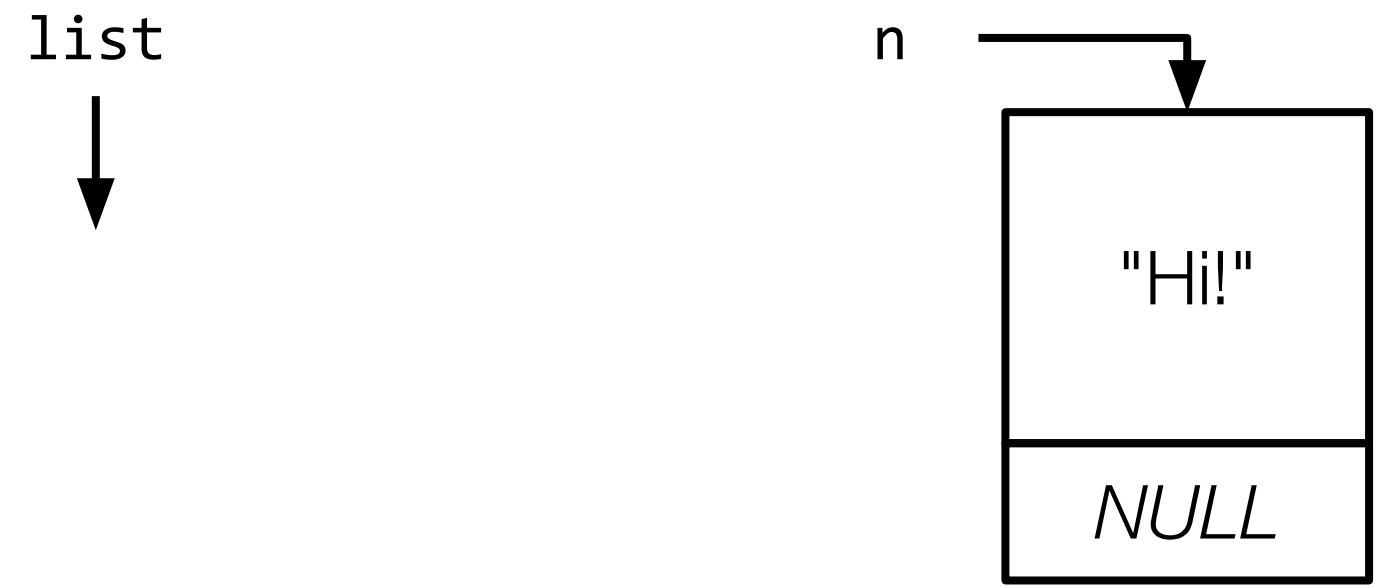
```
node *n = malloc(sizeof(node));
```



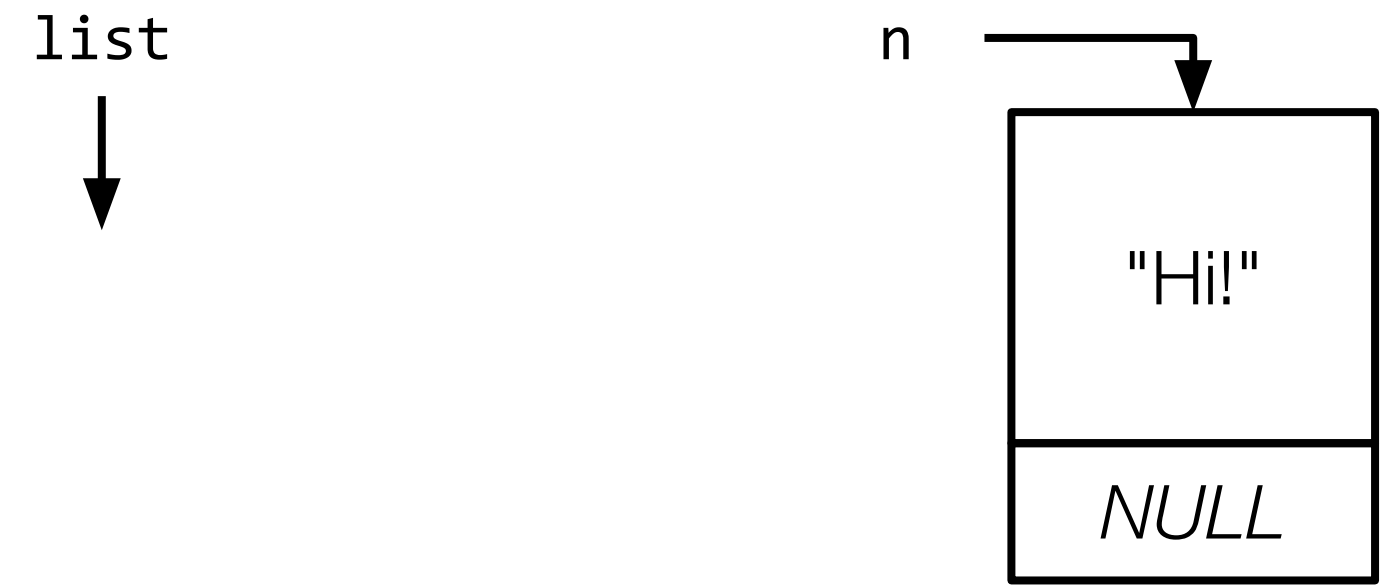
```
node *n = malloc(sizeof(node));
n->phrase = "Hi!";
```



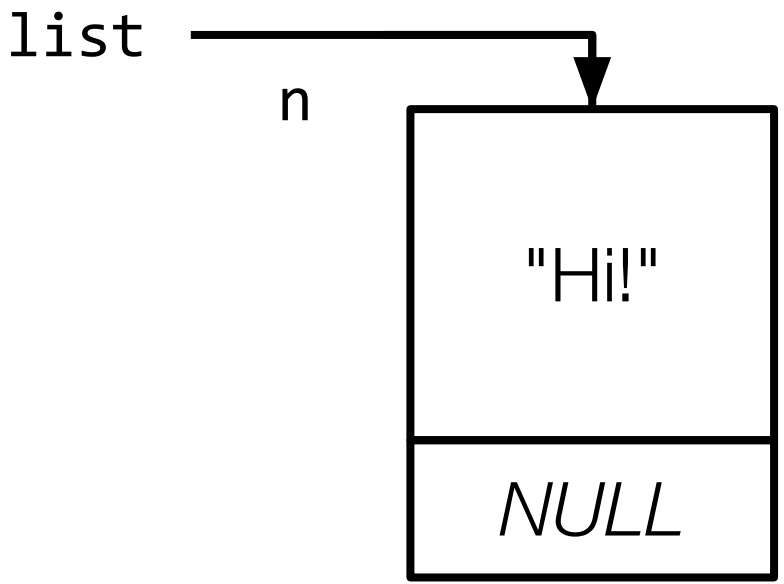
```
node *n = malloc(sizeof(node));
n->phrase = "Hi!";
n->next = NULL;
```



```
list = n;
```

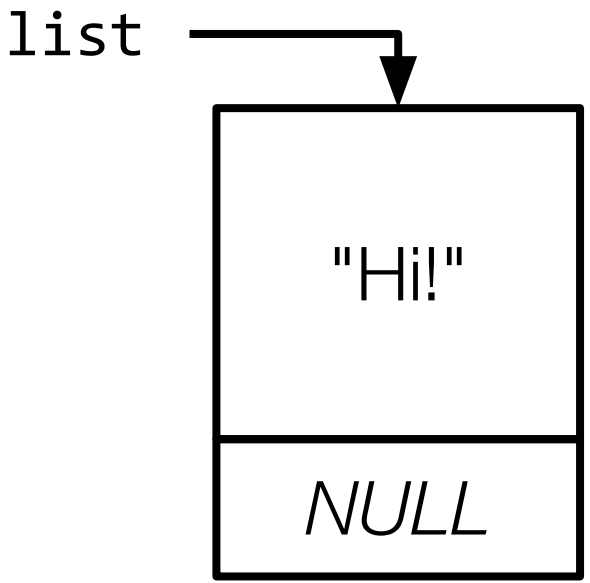


```
list = n;
```

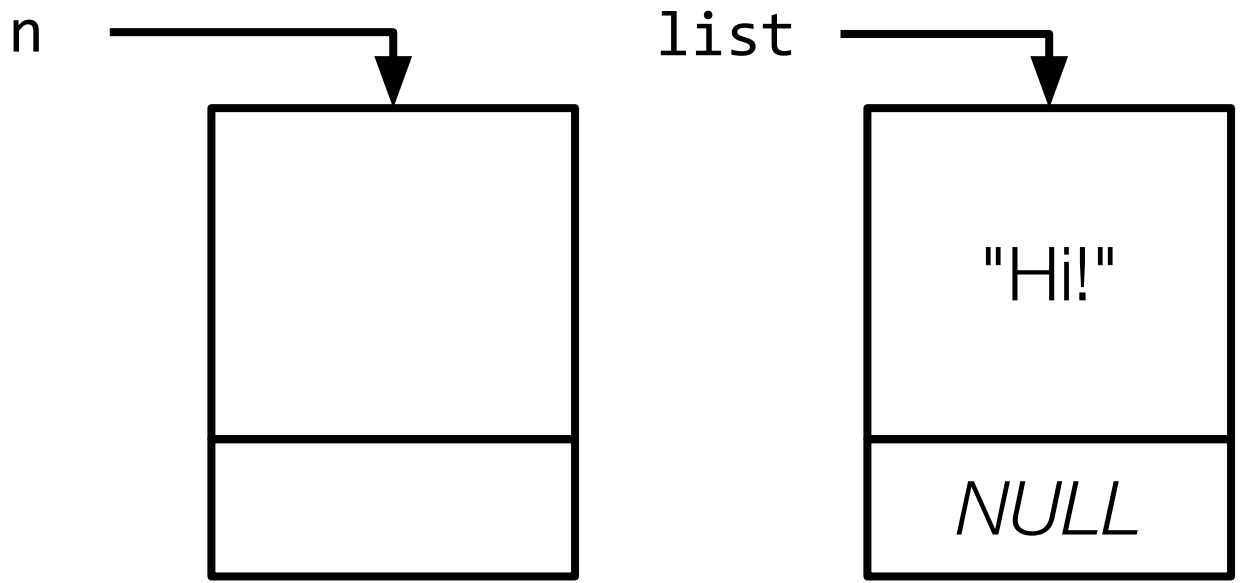


Inserting Nodes

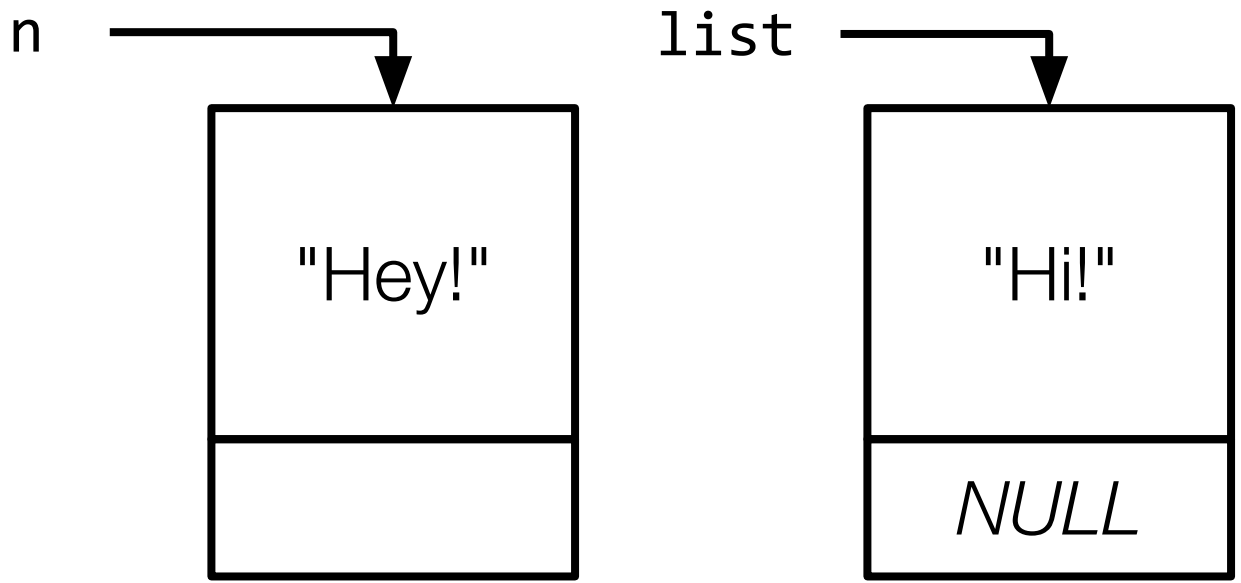
```
n = malloc(sizeof(node));
```



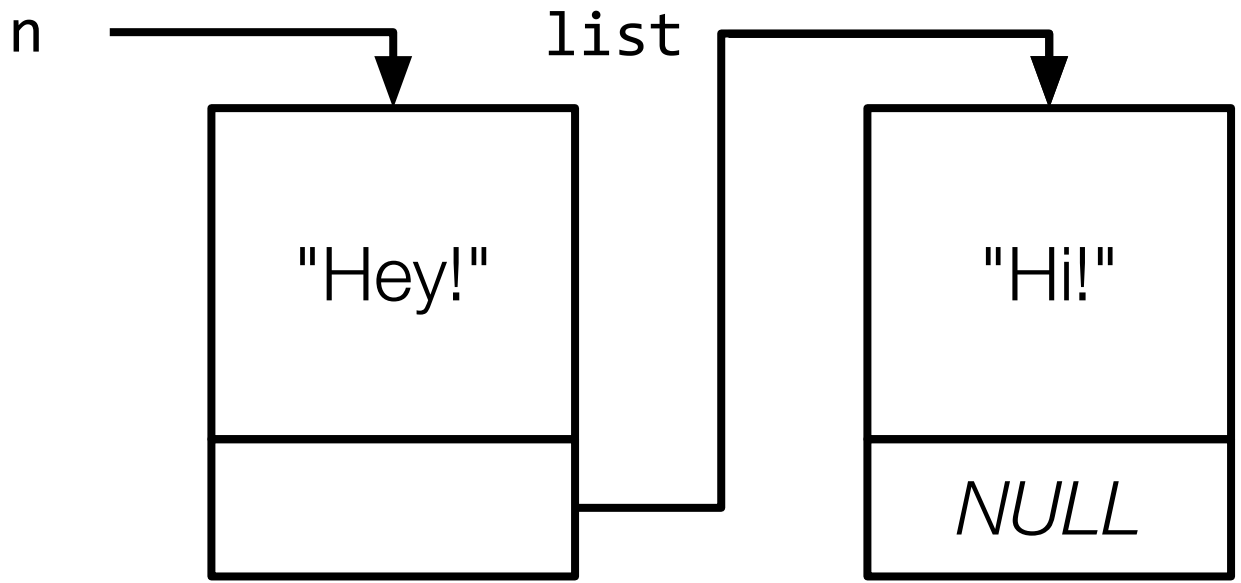
```
n = malloc(sizeof(node));
```



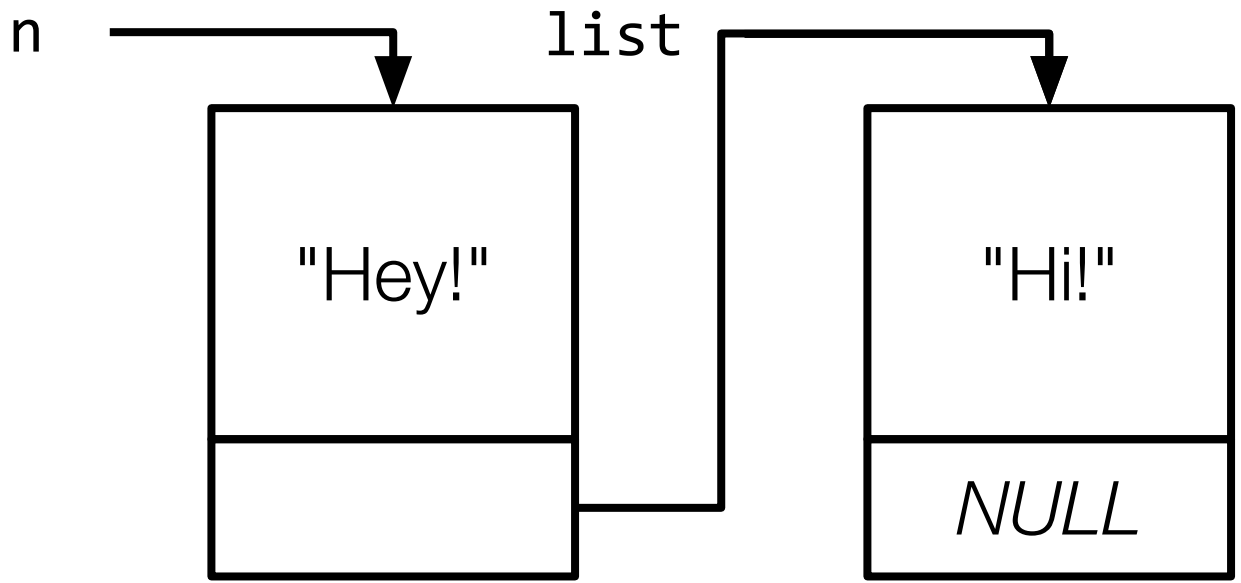
```
n = malloc(sizeof(node));
n->phrase = "Hey!";
```



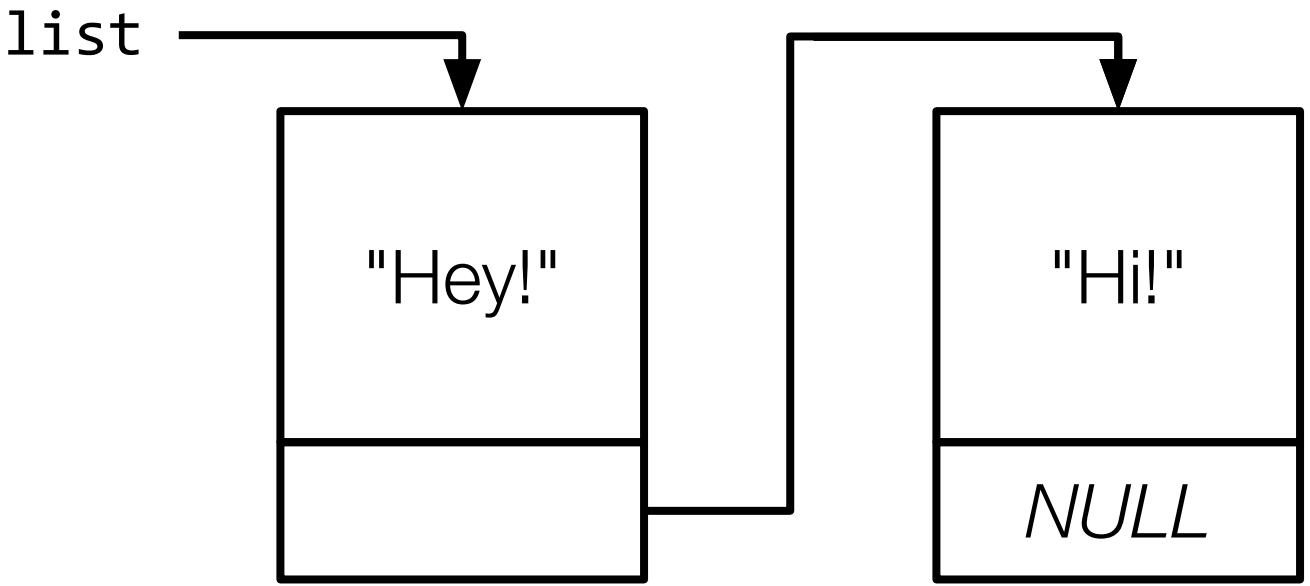
```
n = malloc(sizeof(node));
n->phrase = "Hey!";
n->next = list;
```



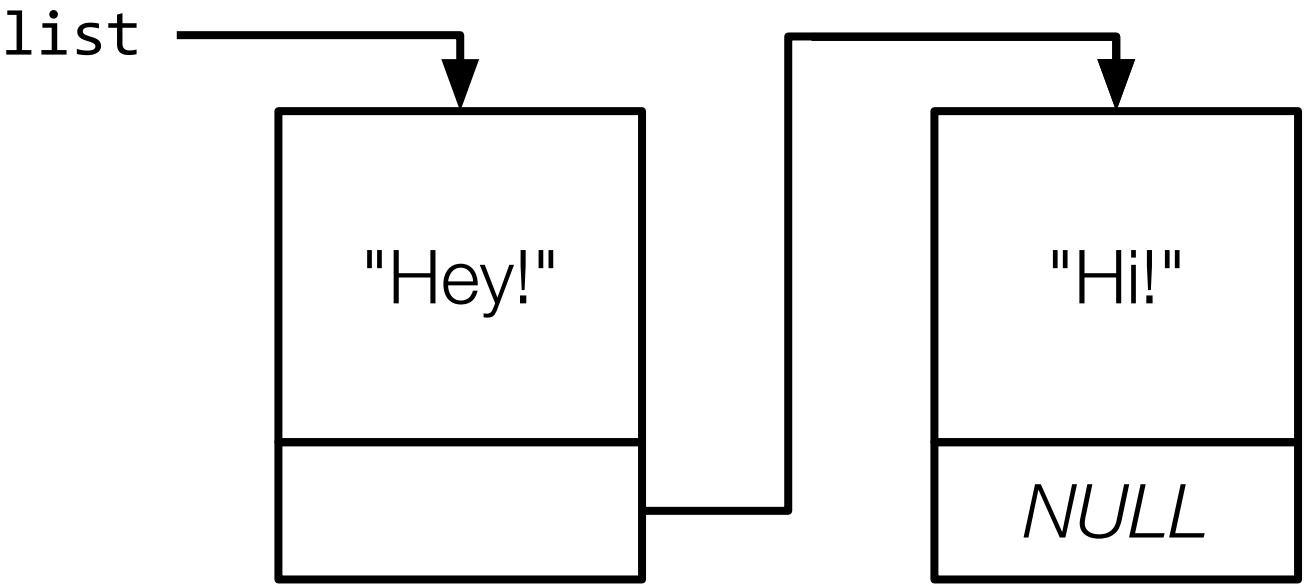
```
list = n;
```



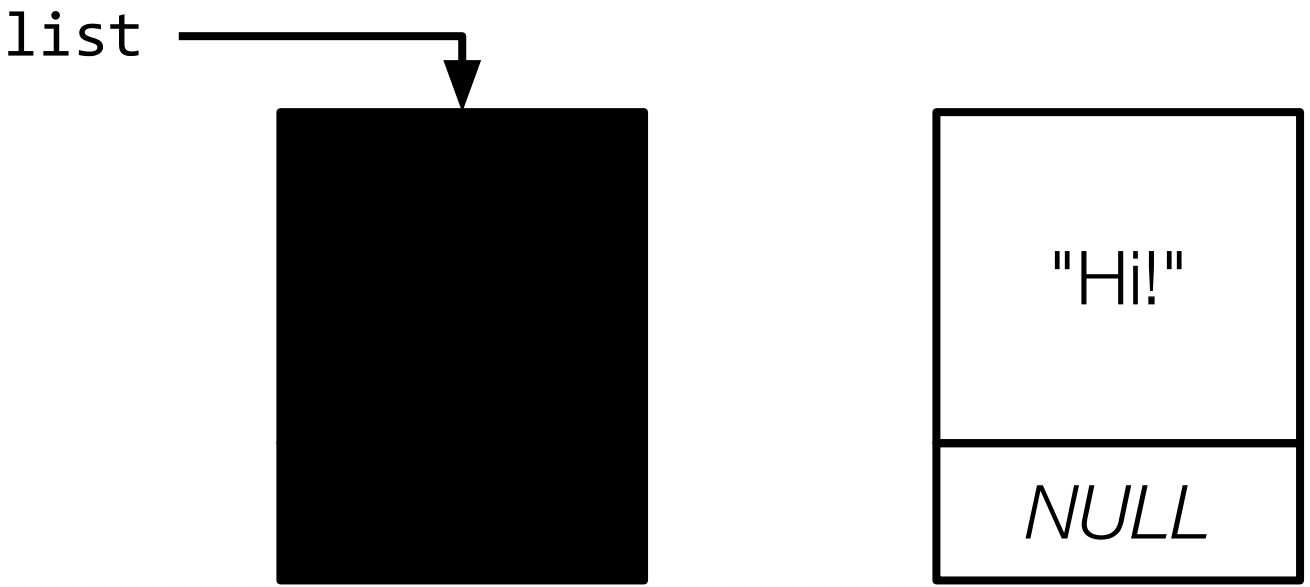
```
list = n;
```



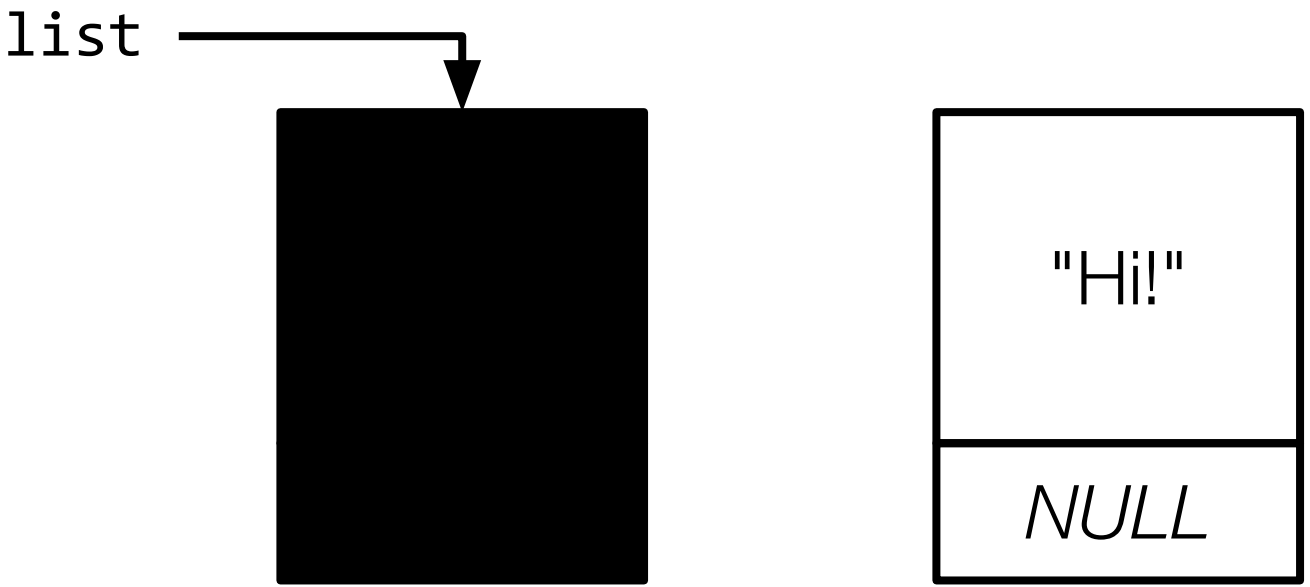
Deleting Nodes

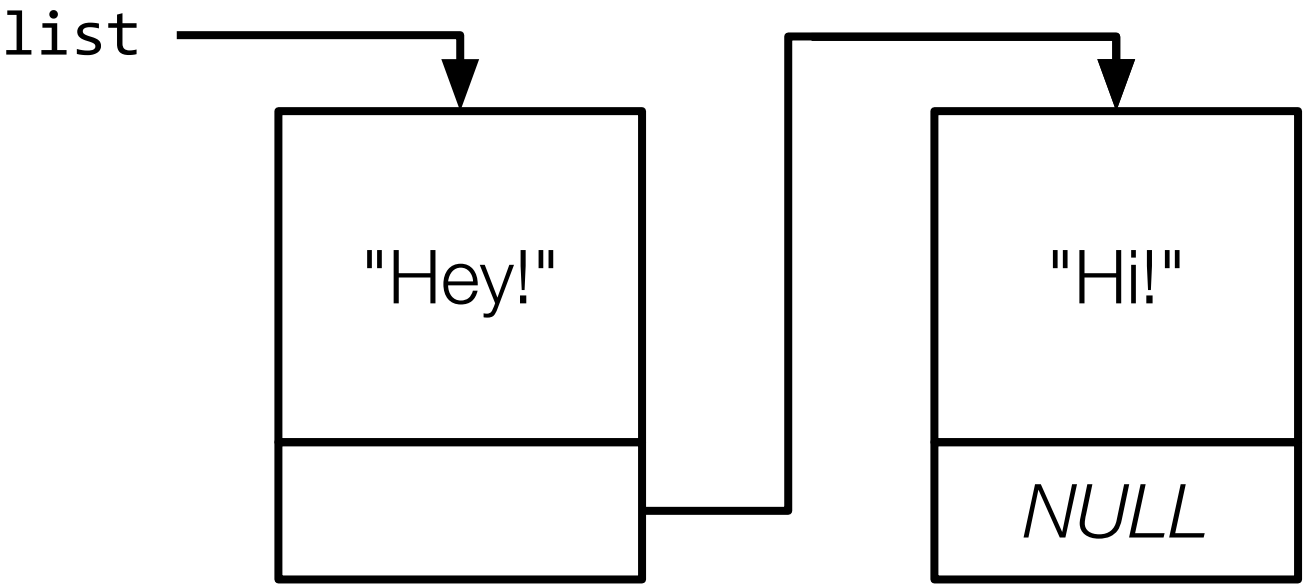

list
"Hey!"
"Hi!"
NULL

```
free(list);
```

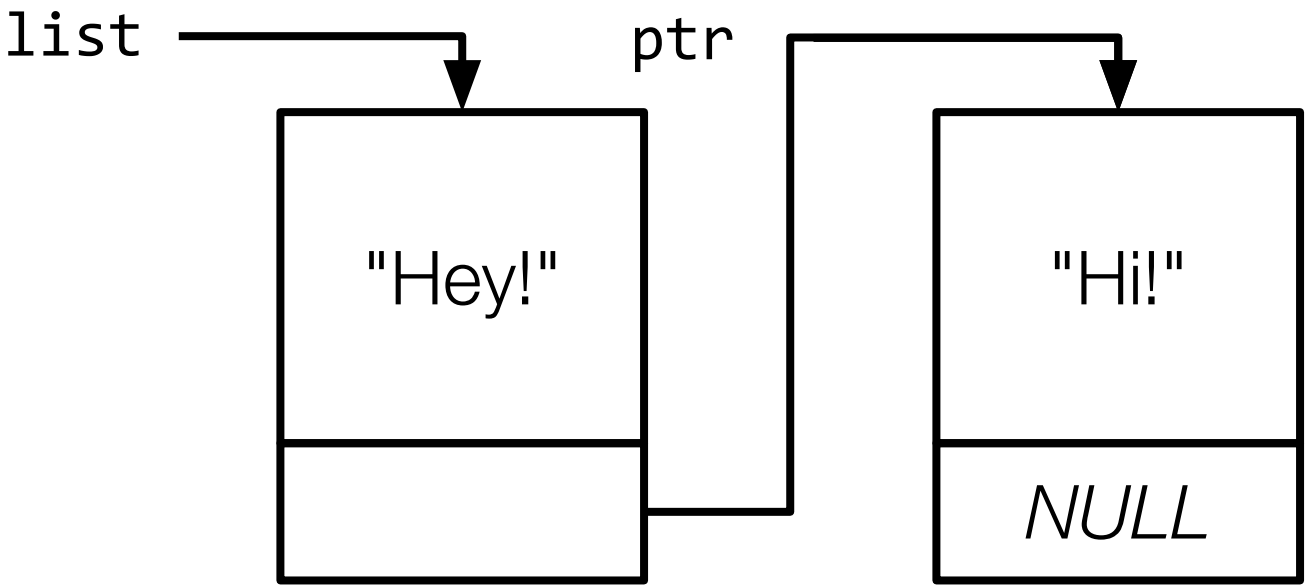


```
free(list);
```

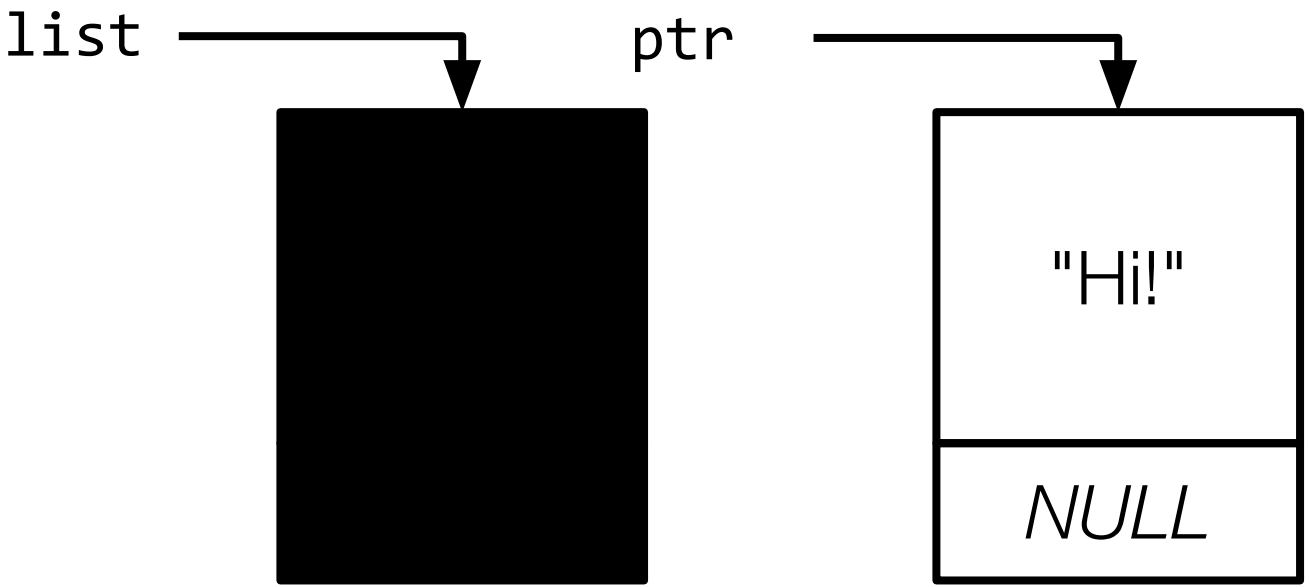



list
"Hey!"
"Hi!"
NULL

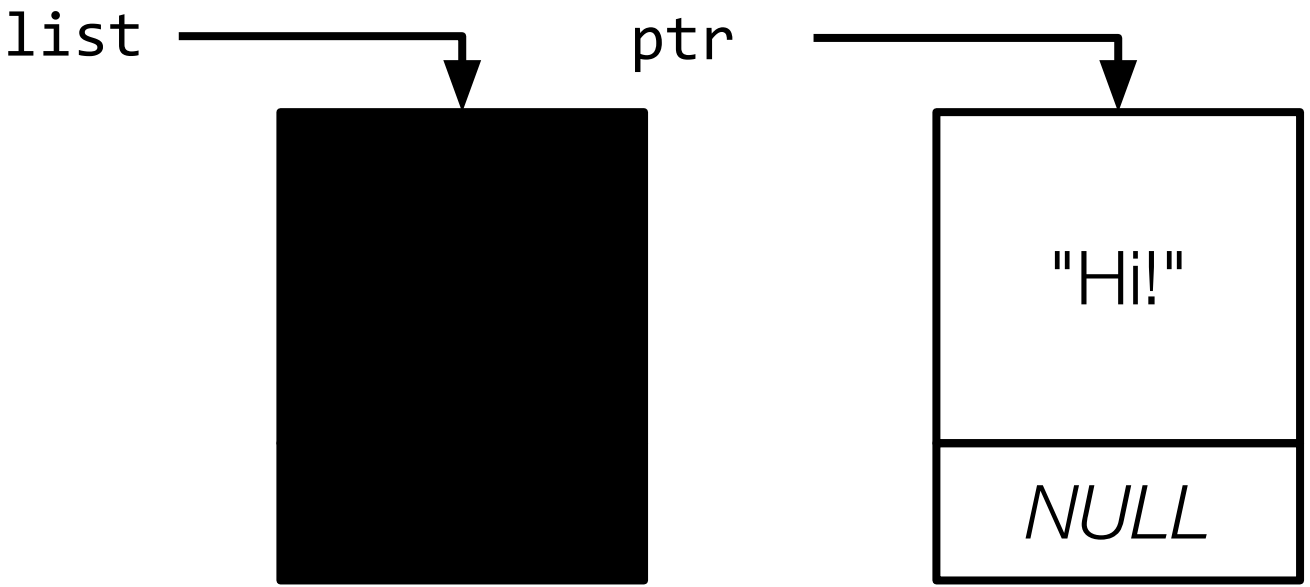
```
node *ptr = list->next;
```



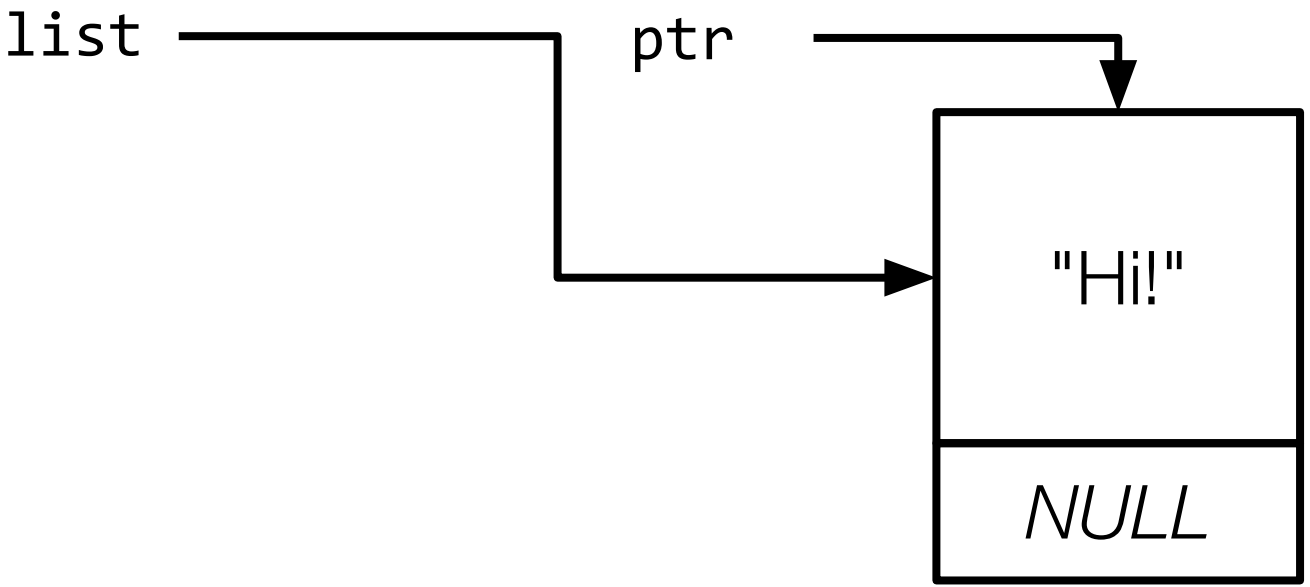
```
free(list);
```



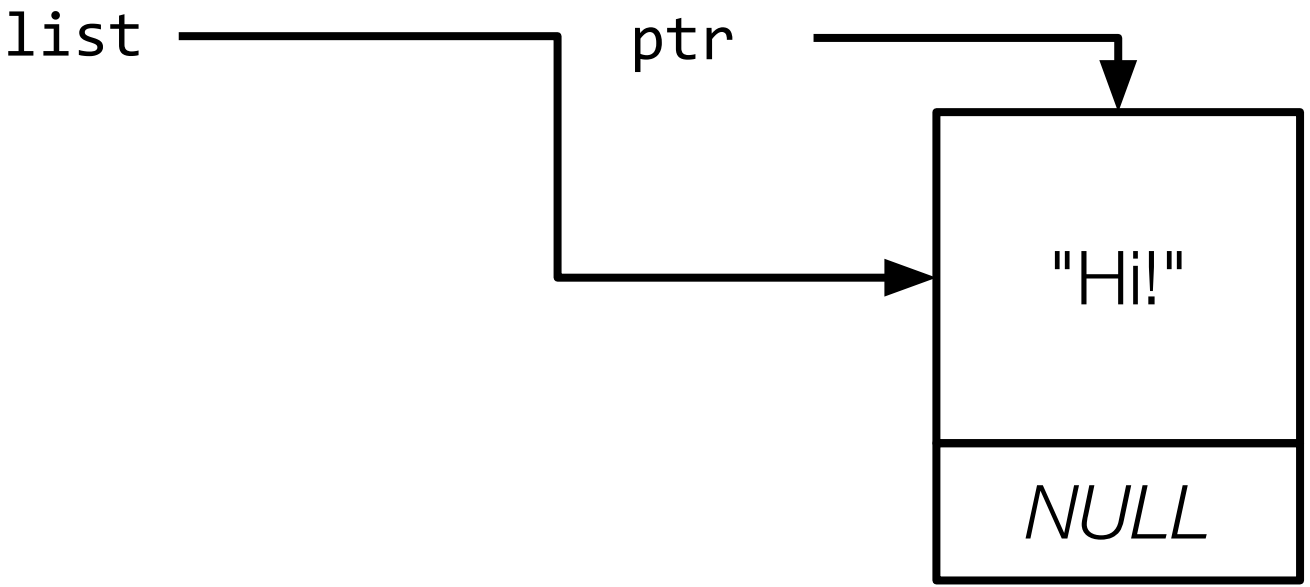
```
list = ptr;
```



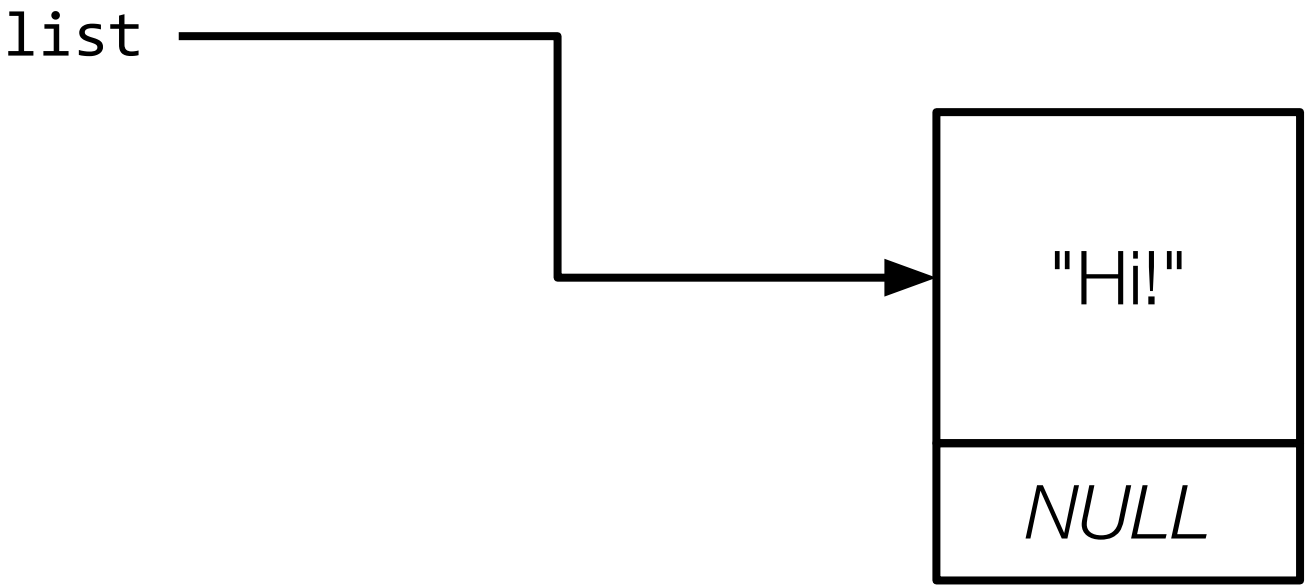
```
list = ptr;
```



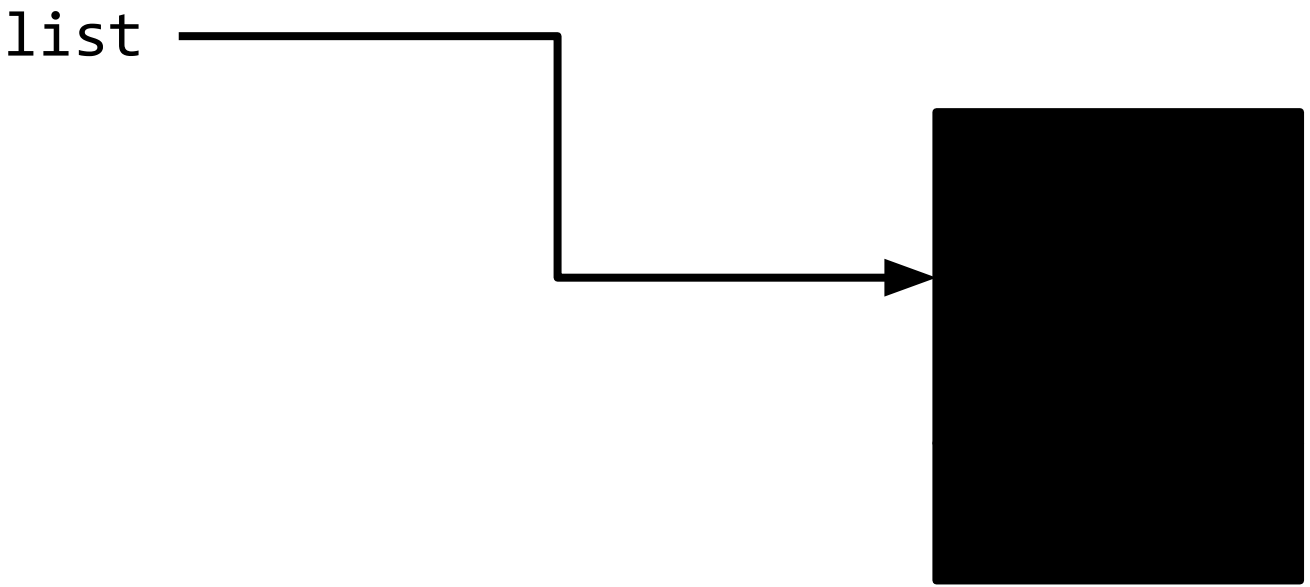
```
ptr = list->next;
```



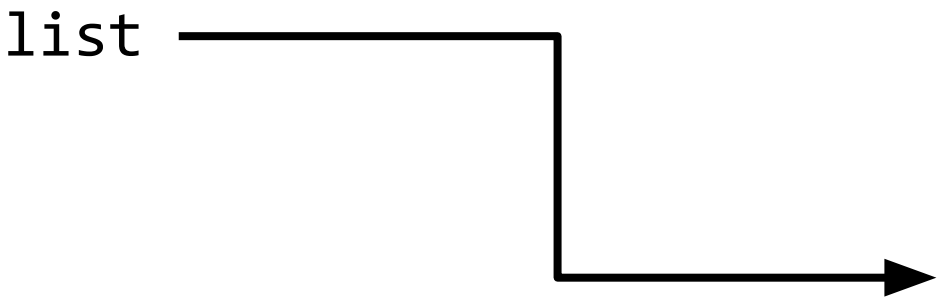
```
ptr = list->next;
```



```
free(list);
```



```
list = ptr;
```

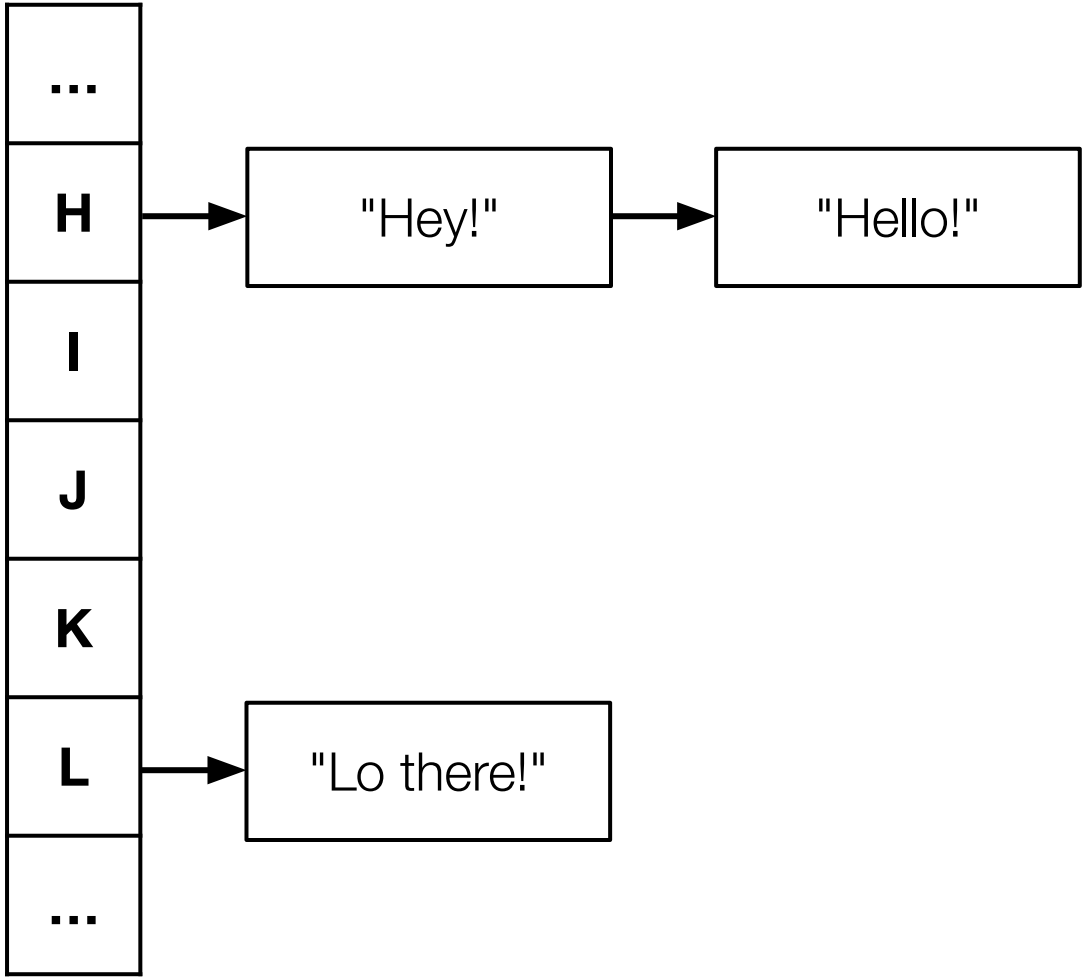


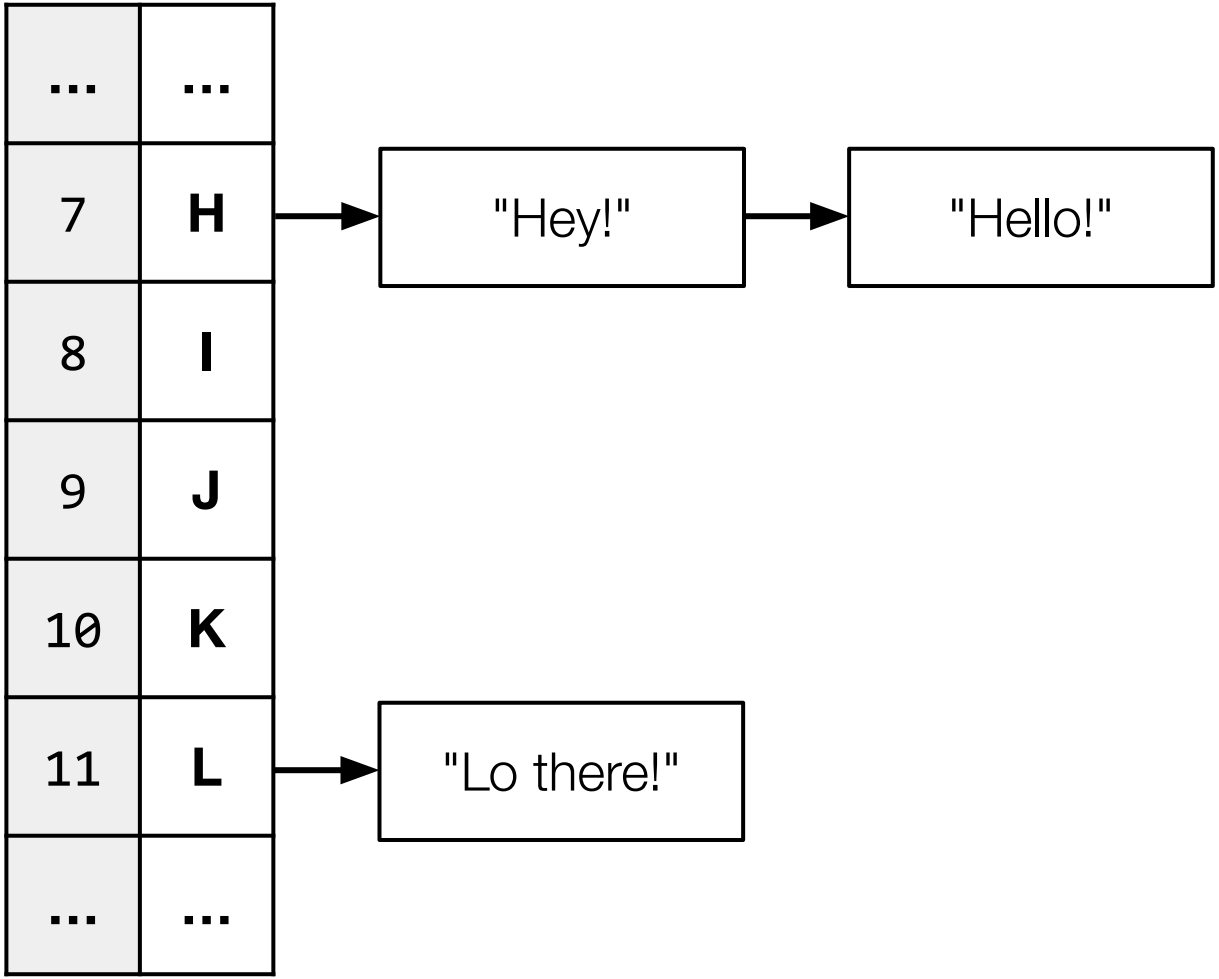
Inserting and Unloading a Linked List

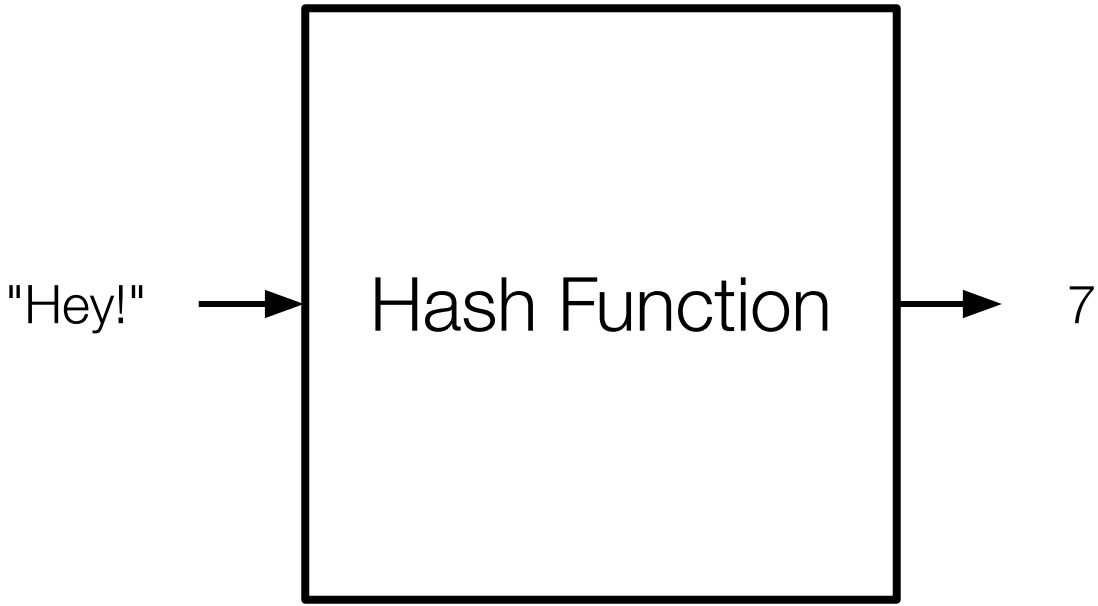
Create a program, **list.c,** where you:

1. implement code to add a node to the linked list. Ensure that **list** always points to the head of the linked list. Also ensure your new node contains a phrase.
2. implement **unload** such that all nodes in the linked list are **free**'d when the function is called. Return **true** when successful.


"Hey!"
"Hello!"
"Lo there!"


...
H
I
J
K
L
...
"Hey!"
"Hello!"
"Lo there!"


...
...
7
H
"Hey!"
"Hello!"
8
I
9
J
10
K
11
L
"Lo there!"
...
...


"Hey!"
Hash Function
7

Hashing

Let’s create a program, **table.c**, where we will complete a hash function to return a number, 0–25, depending on the first character in the word.

A good hash function…

Always gives you the same value for the same input

Produces an even distribution across buckets

Uses all buckets

Inheritance

This is CS50

Week 5